

Spectra — scheda progetto per il colloquio

Samuele Pio Provenzano · agente AI per la chimica, integrato in BioSpecInfo Repository:
samupropio1-ship-it/BioSpecInfo-v11 · demo pubblica su GitHub Pages

In una frase

Un **agente conversazionale** che risolve problemi di chimica e fisica invocando 32 strumenti deterministici invece di rispondere a memoria, legge foto di appunti e PDF, disegna mappe concettuali e risponde al comando vocale — con ragionamento visibile e verificabile, su 10 configurazioni di modello di 7 fornitori diversi, senza alcun backend.

Numeri

Componente	6.840 righe, zero dipendenze runtime
Strumenti	32, su 13 aree scientifiche
Fornitori supportati	7 (Anthropic, OpenAI, Google, DeepSeek, xAI, Z.AI, Groq) — 10 configurazioni, 3 gratuite, tutte attive e raggiungibili dal browser
Dataset interni esposti	9, oltre 800 record
Copertura di test	14 pagine, 84 sezioni, 18 tab — nessun errore JS

Le domande che mi aspetto, e le risposte

«Perché è un agente e non un chatbot?»

Perché il modello **non produce direttamente i numeri**. Quando gli si chiede il pH di un acido debole, invoca un risolutore che risolve l'equazione di secondo grado esatta e restituisce 2,875; il modello commenta il risultato, non lo inventa. Il ciclo di controllo concatena fino a 10 chiamate, e il prompt di sistema impone di verificare l'ordine di grandezza prima di rispondere. La differenza è verificabile: ogni risultato numerico è accompagnato dalla traccia dello strumento che l'ha prodotto, conservata insieme alla risposta.

«Qual è la decisione tecnica di cui vai più fiero?»

Non aver usato `eval()` per il motore di calcolo.

Serviva un valutatore di espressioni matematiche arbitrarie. `eval()` avrebbe risolto il problema in tre righe. Ma l'agente legge contenuti esterni non fidati — risultati PubChem, abstract PubMed, pagine web — e un'iniezione in quei contenuti avrebbe potuto far eseguire codice arbitrario nel contesto della pagina, **dove risiede la chiave API dell'utente**.

Ho scritto un parser a discesa ricorsiva con un insieme chiuso di 27 funzioni. Poi l'ho attaccato: 10 tentativi di evasione (`window.localStorage`, `constructor`, `this`, `fetch(...)`), tutti respinti, verificato anche in un browser reale tentando di leggere davvero la chiave.

È il tipo di scelta che costa mezza giornata e non si vede mai — finché non serve.

«Come hai verificato che funzioni?»

Su tre livelli.

Numerico: ogni risolutore confrontato con valori di riferimento da letteratura — pH 2,875 per l'acido acetico 0,1 M; 55,3 kJ/mol per l'energia di attivazione da due costanti cinetiche; 656,1 nm per la riga H- α ; 8,79 MeV/nucleone per il ^{56}Fe ; t critico 2,7764 per $df = 4$.

Robustezza: casi che *devono* fallire. Equazioni non bilanciabili, formule con parentesi sbilanciate, specie assenti dall'equazione, tentativi di iniezione. Un risolutore che non sa dire "non lo so" è peggio di uno assente.

Integrazione: Chromium headless su tutte le 14 pagine, 84 sezioni percorse una per una, e un ciclo completo simulato con ricerca web, sospensione e ripresa del turno.

Otto bug che ho trovato testando (e cosa insegnano)

1. Il parser di formule sbagliava $\text{Ca}_3(\text{PO}_4)_2$ — dava 215 invece di 310. La prima implementazione gestiva le parentesi con una pila di moltiplicatori scalari, che non regge i gruppi annidati. Riscritta con una pila di mappe di conteggi: ora regge anche $\text{K}_4[\text{Fe}(\text{CN})_6]$. *Lezione: i casi facili passano sempre. Servono i casi difficili scelti apposta.*

2. Dieci dichiarazioni CSS `calc()` silenziosamente invalide. In CSS gli operatori `+` e `-` richiedono spazi su entrambi i lati: `calc(env(safe-area-inset-bottom,0px)+24px)` viene **scartato dal browser**. Gli elementi cadevano nella posizione statica — ed era il motivo per cui il progetto conteneva due watchdog JavaScript che ne forzavano la posizione ogni due secondi. *Lezione: quando trovi una pezza, cerca la ferita.*

3. Il ragionamento del modello veniva catturato ma non mostrato. Il parser dello stream restituiva solo i `text_delta`: con un modello che ragiona, l'utente vedeva una lunga pausa e pensava fosse bloccato. Peggio, il pannello che avevo aggiunto spariva a fine turno, perché la chat viene ricostruita dai messaggi salvati. *Lezione: una funzionalità non è finita quando funziona, ma quando sopravvive al ciclo di vita della UI.*

4. Otto risolutori restituivano ok: `true` con Infinity dentro. Emerso da un audit in cui ho provato i valori limite che in chimica *ha senso* chiedere: lunghezza d'onda 0, volume 0, emivita 0. Il risultato era formalmente valido e l'agente avrebbe riportato « $\lambda_{\text{max}} = \text{Infinity nm}$ » come un dato buono. L'ho corretto con un controllo unico nel punto in cui passano tutti i risultati, invece di rattoppare quattordici casi. *Lezione: un'eccezione si nota, un numero sbagliato che sembra giusto no. Nei sistemi che producono numeri, il fallimento silenzioso è il nemico.*

5. Le miniature riempivano `localStorage`. Salvavo l'anteprima degli allegati in cronologia, ma quell'anteprima era l'immagine intera: 444 KB l'una, dieci foto e la quota era andata. E a quota piena *ogni* scrittura successiva fallisce in silenzio — compresa quella della chiave API.

Era lo stesso guasto che avevo già corretto mesi prima in un altro punto del codice. Ora la cronologia usa una miniatura da 5 KB. *Lezione: quando trovi una classe di bug, cerca altrove prima che ti trovi lei.*

6. Un nome di modello scritto nel codice è una bomba a orologeria. Gemini ha smesso di funzionare da un giorno all'altro: `HTTP 404 – models/gemini-1.5-flash is not found for API version v1beta`. Google aveva ritirato il modello, e l'URL lo conteneva alla lettera. La correzione ovvia era scriverci il nome nuovo: avrebbe funzionato fino al ritiro successivo.

L'API però sa dire quali modelli esistono *per quella chiave*. Ora il nome non c'è più: `ListModels` restituisce il catalogo, un punteggio sceglie il migliore — versione più recente, famiglia `flash`, scartando varianti sperimentali e modelli non conversazionali (audio nativo, immagini, embedding) — e la scelta resta in cache per una settimana, legata a un'impronta della chiave perché chiavi diverse vedono cataloghi diversi. Se `ListModels` non risponde si sondano dei candidati noti con una GET sui metadati, che non consuma quota di generazione. E se il modello viene ritirato *mentre* è in cache, il 404 della chiamata vera invalida la cache e fa ripartire la ricerca (vedi il bug 8: la prima versione ritentava una volta sola, e non bastava).

E poi è successo di nuovo, su un altro fornitore. Groq ha ritirato `llama-3.3-70b-versatile` il 17 giugno, e l'app si è fermata allo stesso modo: avevo corretto Gemini e lasciato la stessa bomba in Groq, OpenRouter e xAI. La seconda volta ho generalizzato invece di ripetermi — con una strategia diversa per famiglia, perché i nomi lo sono: su Gemini si può ordinare per versione, sui fornitori OpenAI-compatibili no (`openai/gpt-oss-120b` contro `qwen/qwen3.6-27b` non si confrontano), quindi l'ordine dei candidati è la preferenza e l'elenco serve solo a saltare quelli spariti.

Verificato su 51 casi con l'API simulata: «esce Gemini 3.0» (lo sceglie da solo), «il candidato preferito sparisce» (scala al successivo), «nessun candidato sopravvive» (punteggio sui disponibili, scartando trascrizione e classificatori), «rete giù», «ritiro a caldo» (risolve e ritenta una volta) e «404 persistente» — che deve fermarsi, non entrare in ciclo.

Lezione: quando un guasto verrà di sicuro di nuovo, la correzione giusta non è il valore nuovo, è togliere il valore. E quando lo correggi, cercalo subito in tutti i posti dove vale — io la prima volta non l'ho fatto.

7. «Il servizio è attivo» non vuol dire «funziona». Avevo controllato uno per uno che tutti gli undici modelli fossero ancora vivi. NVIDIA NIM lo era: servizio attivo, piano gratuito, chiave valida, modello giusto nel catalogo. E dal browser dell'utente non partiva una chiamata.

Perché avevo verificato la dimensione sbagliata. Una pagina statica chiama i fornitori **dal browser**, e il browser lascia partire la richiesta solo se il fornitore manda gli header CORS — una proprietà indipendente dall'essere vivo, che ogni fornitore decide da sé e cambia senza annunci. Peggio: quando la richiesta viene bloccata il browser *non dice perché*. Rete assente, CORS negato e servizio spento arrivano come lo stesso `TypeError: Failed to fetch`.

La correzione ovvia era cercare quali fornitori supportano CORS e scriverlo in un elenco. Sarebbe stata la terza volta che scrivo un valore destinato a scadere — e per giunta un valore che non è nemmeno lo stesso per tutti, perché dipende anche dalla rete e dalle estensioni di chi usa l'app.

Così la risposta non la do io: la **misura l'app**. Ogni fetch fallito annota il fornitore con data e conteggio; ogni risposta HTTP ricevuta — anche un 401, che dimostra di essere arrivati al server — cancella l'annotazione. Gli annotati prendono un ▲ nella tendina, scendono in fondo alla catena di riserva e non vengono più scelti dalla modalità ad alta potenza, ma **non spariscono**: dopo 24 ore si ritentano da soli, perché una rete giù per un minuto non è un servizio incompatibile, e se è l'unico fornitore configurato va provato comunque.

Verificato su 70 casi, di cui 35 in un browser vero con la rete negata a quel solo dominio: l'annotazione compare, il ▲ appare senza ricaricare la pagina, il secondo fallimento cambia il messaggio da «può essere la rete» a «scegli un altro fornitore», e un 401 finto la fa sparire.

Lezione: verificare la proprietà giusta. «Esiste» e «funziona nel contesto in cui gira davvero» sono due domande diverse, e avevo risposto solo alla prima. Quando nemmeno la seconda ha una risposta stabile, la cosa onesta non è indovinarla: è costruire il sistema che la misura da sé.

8. Il ritentativo c'era, e non poteva funzionare. Il meccanismo del bug 6 sembrava chiuso finché Google non ha risposto questo a una chiave valida:

```
> This model models/gemini-2.5-flash is no longer available to new users. > Please update  
your code to use models/gemini-3.6-flash
```

Non «ritirato»: **non disponibile per i nuovi utenti**. Il modello esisteva ancora — `ListModels` lo elencava, la GET sui metadati rispondeva 200 — e solo la chiamata di generazione dava 404. Tutte le mie verifiche guardavano l'esistenza; l'esistenza non era la proprietà giusta.

Il guasto vero però era un altro, ed era mio. Il codice, preso il 404, invalidava la cache e risolveva. Ma **la ri-risoluzione è deterministica**: stesso catalogo, stesso punteggio, stesso nome. La condizione `nuovo !== vecchio` era falsa e l'app si arrendeva. Avevo scritto un ritentativo che non poteva riuscire, e nessun test lo aveva mostrato perché tutti i miei casi presupponevano che il modello sostitutivo fosse *nel catalogo*.

Tre pezzi, e servono tutti e tre. **Il nome fallito viene bocciato** e escluso dalle risoluzioni successive: è questo a rendere ogni tentativo diverso dal precedente, cioè a far *terminare* la ricerca invece di girare a vuoto. **Il sostituto si legge dal messaggio del fornitore** — «use models/gemini-3.6-flash» — che è la fonte migliore possibile: viene da lui, è aggiornata all'istante e non può scadere come un valore che scriverei io. E fra i candidati di riserva ora c'è per primo un **alias** (`gemini-flash-latest`), che per costruzione non viene mai ritirato.

Verificato su 36 casi, fra cui i due che contano: il turno che prima si fermava adesso *riesce* usando il modello che il fornitore stesso ha indicato, e il caso in cui non funziona niente termina senza mai ripetere due volte lo stesso nome.

Lezione: un meccanismo di recupero va testato sul caso in cui deve scattare, non solo su quello in cui è comodo simularlo. Il mio girava a vuoto da mesi e sembrava a posto. E quando un sistema esterno ti dice cosa fare, ascoltarlo batte qualsiasi euristiche che puoi scrivere tu.

Competenze dimostrate

- **Integrazione LLM in produzione:** function calling multi-fornitore, parsing di stream SSE, gestione dei blocchi di ragionamento con firma, ripresa di turni sospesi lato server, configurazione per-modello dei parametri API, ingresso multimodale (immagini, PDF, testo) con codifica per famiglia.
 - **Sicurezza applicativa:** modello di minaccia dell'iniezione via contenuti esterni, superficie di esecuzione ridotta al minimo, gestione della quota di storage come vettore di guasto silenzioso.
 - **Metodo di verifica:** casi di riferimento da letteratura, casi negativi espliciti, test d'integrazione in browser reale automatizzato.
 - **Dominio scientifico:** 13 aree fra chimica, chimica fisica, biochimica, farmacologia, fisica nucleare e astrofisica, implementate come risolutori esatti e validate.
 - **Front-end senza dipendenze:** renderer di grafi SVG con algoritmo di layering e riduzione degli incroci, parser Markdown con tabelle, comando vocale con parola di attivazione, esportazione in PDF.
-

Limiti che dichiaro apertamente

La chiave API risiede nel browser: dentro una pagina statica non c'è alternativa. Per l'uso condiviso ho scritto un Worker Cloudflare che tiene le chiavi lato server — con subentro automatico fra più chiavi quando una esaurisce la quota — ma resta un componente in più da mantenere, e i suoi limiti per IP sono in memoria, quindi approssimativi: frenano l'abuso, non lo azzerano. I risolutori usano modelli semplificati dove la letteratura ne ha di più raffinati, e ogni strumento dichiara il metodo che ha usato. La verifica è funzionale e numerica, non formale: non c'è prova di correttezza, c'è un insieme di casi di riferimento.

Documentazione completa: [docs/05-AI-Agent-Architecture.md \(IT\)](#) · [docs/en/05-AI-Agent-Architecture.md \(EN\)](#)